

## Lab 8

## Objective:

- Calling functions in x86-64 assembly language and managing the stack and each function's stack frame.

The purpose of this lab is to examine the x86-64 function call mechanisms in action. Given a program, i.e., a collection of functions that call each other, you will identify variable allocation — both in registers and on the stack (i.e., stack frame) — and draw the resulting stack, as well as the register table, as this program executes (i.e., as you are hand tracing it).

## Part 1: Getting Started

- Login to the CSIL machines in the Linux environment, download and unzip **Lab8-Files.zip** into **sfuhome/CMPT295**, and change directory to **Lab8** directory listing should reveal a **makefile** and the source files **main.c**, **p1.c** and **p2.c**.
- Start by opening **main.c** in your favourite editor. The program contains three variable declarations: a 40-byte **char** array and two 4-byte **ints**. It displays the original values of two of these variables, and then calls the function **proc1()**, which is defined in **p1.c**. The parameters of **proc1()** are memory addresses (pointers) to the variables **buf**, **x** and **y**. This will allow the callee to modify the content of these variables, which the subsequent calls to **printf()** and **puts()** will confirm.
- Now open **p1.c**. It uses the two local variables **v** and **t**, calls **proc2()** (which is defined in **p2.c**), re-computes the values of **s**, **a** and **b**, and returns.
- Make sure you understand what the code in these three files does.
- On the command line, run **make** and then execute the program. Make sure you agree with its results.

Part 2: **main()** calling **proc1()**

Your task in this part of the lab is to draw the stack diagram (or stack segment of the memory) that is produced as **main()** executes and calls **proc1()**. To do so, use the Stack Diagram found on the last page of this lab.

You must do this by hand tracing the assembly code you produced in Part 1 of this lab (when you ran **make**), starting with the assembly code of the **main()** function until it calls **proc1()**, including the **call** instruction itself then stop!

Model your drawing of the stack diagram based on the way we have been drawing our stack diagrams in our Lectures 20 and 21.

**Requirement:** For this Lab 8, make use of the **register table** (which is part of the Stack Diagram found on the last page of this lab) as you hand trace the code, listing all the registers used so far and their values. This register table will help you deal with the ever-changing content of the registers. You may find this table helpful.

#### Instructions to help you draw the stack diagram:

- Open **main.s**, the caller. This file contains a lots of directive statements (these are the statements starting with a period) and assembly code instructions. The first assembly code instruction of **main** we shall start with is **pushq %rbp** at Line 18. The effect of this instruction on the stack has already been illustrated (drawn) on our stack diagram on the last page of this lab. Have a look! As you can see, the content of the register **%rbp** has already been pushed at the top of the stack. Since we do not know the value stored in this register, we shall simply use the name of the register to indicate this value.

Question: Why is **main** pushing the content of **%rbp** on the stack? Indicate your answer under the column named **Purpose** on the right-hand side.

At this point in the semester, you should be able to answer this question. And, as you step through the assembly instructions, it will certainly become clear to you.

- On Line 21 and 22, our local variables **x** and **y** are being initialized, i.e., their initial values are loaded onto registers **%edx** and **%ecx**, respectively.
- On Line 23 and 24, **main** is preparing the arguments of the function **printf(...)** which it calls on line 33.
- At line 25, 64 (8 x 8 bytes) is subtracted from the current stack pointer **%rsp**. This has the effect of reserving 64 bytes of memory space for **main()** on the stack. So, on the stack diagram, mark these 64 bytes, or 8 quad words (i.e., 8 rows), as “space for **main()**’s local variables + others” under the column named **Purpose**.
- The effect of subtracting 64 bytes from the stack pointer **%rsp** (i.e., from the memory address the stack pointer **%rsp** contains) is reflected on the stack diagram by moving the

label “**%rsp**”. This has already been done for you on the diagram. Indeed, this location has been labelled “**%rsp + 0<sub>10</sub>**” (instead of simply this “**%rsp**”). Why? Let’s explain:

- Addresses within the “space for **main()**’s local variables + others” are expressed, on this stack diagram, relative to the current value (or location) of the stack pointer **%rsp**. This is to say that the current location of the stack pointer **%rsp** (i.e., the top of the stack) is 0<sub>10</sub> (0 byte) away from its current location on the stack.
  - This format “**%rsp + 0<sub>10</sub>**” is composed of a **base register** (here: **%rsp**) to which a **displacement** or offset (i.e., a factor of 8 bytes) is added. So, the next quad word within the “space for **main()**’s local variables + others” will have the memory address **%rsp + 8<sub>10</sub>** (displacement of +8 bytes from the current stack pointer **%rsp**), and the next will have **%rsp + 16<sub>10</sub>** (displacement of +16 bytes – 2 x 8 bytes - from the current stack pointer **%rsp**), and so on.
  - Fill in all the other labels (i.e., “**%rsp + displacement value**”) to the left of each stack cell (or row) within **main()**’s stack frame. There are already three of them on the diagram (**%rsp + 0<sub>10</sub>**, **%rsp + 8<sub>10</sub>**, **%rsp + 16<sub>10</sub>**). There should be total of 8 such labels. Fill in the missing five (5).
- On lines 27 and 28 of **main()**, the instruction pair

```
movq %fs:40, %rax
movq %rax, 56(%rsp)
```

loads the **canary value** to detect **buffer overruns**. Therefore, write “canary value”, in the stack at **%rsp + 56<sub>10</sub>**.

We will talk about **canary value** and **buffer overruns** in our lectures very soon.

- We know what is happening at Line 29, don’t we?
- As a local variable, the array **buf** (`char buf[40]`) is stored on the stack. Have a look at Line 30. Remember that an array can also be seen as a memory address (the memory address of the array’s first byte). Also, recall that the **leaq** instruction computes a number and that this number can be interpreted as a memory address, but this **leaq** instruction does not dereference it (i.e., it does not access memory – here the stack). Place **buf** and its 40 cells in their appropriate stack location. Make sure you clearly indicate where **buf** starts and finishes on the stack and label all these rows “**buf**”.
- Are the values of the local variables **x** and **y** also stored on the stack? To answer this question, look at Line 31 and Line 32. Where on the stack are they stored? Remember, **x**

has already been initialized to the value **6** and loaded into **%edx** and **y** to the value **9** and loaded into **%ecx**. Place these variables (names and values) on the stack diagram in the **correct** location.

- After calling **printf()** at line 33, the three instructions that follow are the setup for calling function **proc1()**. Remember that the x86-64 function call protocol dictates that the first argument be placed in **%rdi**, the second in **%rsi** and the third in **%rdx**. And again, recall that the **leaq** instruction computes a number which can be interpreted as a memory address, but does not dereference it. Thus, each of **%rdx**, **%rsi** and **%rdi** contain memory addresses, i.e., they are pointers. Here are examples of how we create pointers using the stack.
- The call to **proc1()**, at Line 37, pushes the return address (address of assembly instruction at Line 38), thus ending the stack frame for the caller **main()**. Add this return address to your stack diagram. For the purpose of this lab, simply write in the appropriate stack cell (or row) the words “return address for **main** at Line 38”. Notice that this created an empty 8-byte row on the stack.
- As we saw in our lecture, a stack frame is the section of the stack segment utilized by a function as it executes. To clearly indicate the extend of the stack frame for the function **main()**, i.e., where it ends, draw a thick line just “below” this stack frame. Also, under the column named **Purpose** on the right-hand side, label this stack frame using the name of its associated function. Note that, in terms of **main()**, we have already done so.
- At this point in our Lab 8, you may wish to have the TA verify your stack diagram.

### Part 3: **proc1()** calling **proc2()** and **proc2()** itself.

---

Your task for this part of the lab is to draw the stack frame for **proc1()** until it calls **proc2()** and the stack frame for **proc2()** if there is one allocated, i.e., if **proc2()** actually uses the stack.

**Important:** Your final stack diagram and register table should reflect the execution of the **main()**, **proc1()** and **proc2()** functions up until **proc2()** has reached its **ret** instruction, **but has not yet executed it.**

Bottom line: We are hand tracing the code of **main()** up until and including Line 37 in **main.s** and we are hand tracing the code of **proc1()** up until and including Line 35 in **p1.s** and we are hand tracing all the code of **proc2()** up until and including Line 11.

The following instructions will help you with this task:

- Open `p1.s`, and have a look at `proc1()`. Within its first few lines (Line 15 to Line 29 – skip the assembly directives, i.e., statements that start with a period), it saves four registers on the stack (on its stack frame). Why are they being saved? Hint: Are they *callee saved* or *caller saved* registers? Add them to your stack diagram and add a note under the column **Purpose** describing what `proc1()` is doing.
- Where are the variables for `proc1()` stored? Hand trace the code on Line 21, Line 25, Line 29 and Line 32 to Line 34, just before the call to `proc2()` at Line 35, and deduce which registers hold which values. Update the register table as you go. In other words, which register is holding each of the following values:
  - `char *s` (the pointer)
  - `int *a` (the pointer)
  - `*a` (the dereferenced pointer)
  - `int *b` (the pointer)
  - `*b` (the dereferenced pointer)
  - `v` (the integer result of `proc2()`)

The variable `t` wasn't actually necessary. The compiler optimized it away.

- Make sure you hand trace Line 30 and Line 35, modifying the stack as necessary, then draw a thick line just “below” `proc1()`'s stack frame to clearly indicate where it ends. Also, under the column named **Purpose** on the right-hand side, label this stack frame using the name of its associated function.
- Hand trace `proc2()` in `p2.s` and modify the stack diagram and the register table accordingly. In which registers are `proc2()`'s local variables `m` and `n` found?
- Make sure you keep updating the **register table** as you hand trace the code, listing all the registers used so far and their values.
- Again, at this point in our Lab 8, you may wish to have the TA verify your stack diagram.

## Challenge

---

There are two things you should experiment with, both tied to the size of the array `buf` (i.e., string `s`)

1. If you reduce the size of the array **buf** (i.e., string **s**) from **buf[40]** to say, **buf[39]** and re-make the code, you'll notice that **main.s** did not change. This is probably because of word boundary optimizations: it is more efficient for the machine to access the canary value (and perhaps other 8-byte values as well) if these values do not cross a word boundary, i.e., if they are stored in stack cells with addresses that are multiple of 8. Thus, the effective size of **buf** (and other oddly sized data types) will usually get rounded up to the nearest multiple of 8.
2. But here is the really strange thing: if you reduce the size of the **buf** to 32, there will still be no change in **main.s**. Try it! This has to do with stack optimization. For some portion of the instruction set that deals with streaming applications (see the Aside at page 276 in our textbook), it is critical that the stack pointer be a multiple of 16. This is called *stack alignment*, and it is not a big deal for the applications we are pursuing. However, because the compiler pays attention to stack alignment issues, you will not see any effect on **main.s** until you reduce the size of **buf** to < 25.
3. Try it by reducing the size of **buf** to 24 and re-making the code. Then have a look at **main.s**. Notice that it now allocates a stack "space" of 48 bytes instead of the original 64 bytes, i.e., a difference of 16!

Lastly, re-run your modified program (the one with **buf[24]**) and observe an unusual program exception. Hum... What do you suppose happened here?

Again, we will talk about this interesting situation in our lectures very soon.

Stay tuned!

Student number:

## Lab 8 – Participation Activity 5

[illegible]

### Register Table :

[illegible]